

FovRL: Joint Foveation and Quality Control for Immersive VR Streaming Using Reinforcement Learning

Yuk Hang Tsui*
Hong Kong University of Science and
Technology
Hong Kong, Hong Kong

Ze Wu*
Hong Kong University of Science and
Technology
Hong Kong, Hong Kong

Ahmad Alhilal*
Aalto University
Espoo, Finland

Matti Siekkinen
Aalto University
Espoo, Finland

Pan Hui†
Hong Kong University of Science and
Technology (Guangzhou)
Guangzhou, China

Abstract

VR cloud gaming promises immersive experiences, yet its realization is critically challenged by the trade-off between stringent latency requirements and high visual quality under unpredictable network conditions. Existing heuristic adaptive bitrate and foveation approaches lack adaptability to highly dynamic mobile networks. This results in a suboptimal trade-off between bandwidth usage and visual quality. While data-driven approaches (i.e., reinforcement learning, RL) have been successful in video streaming, their application to VR cloud gaming poses particular challenges. The stringent demands for high resolution and frame rate, and ultra-low latency are compounded by the necessity for fine-grained, per-frame inference to adapt to rapid changes in user gaze and network conditions. This work introduces FovRL, an RL framework for jointly optimizing foveation parameters and bitrate allocation in response to real-time network throughput. Our work pioneers the application of RL for real-time foveated encoding in immersive VR cloud gaming. Evaluations over real-world networks reveal that FovRL enhances bitrate adaptability to deliver superior perceptual visual quality, while maintaining latency comparable to the SoTA.

CCS Concepts

• **Computing methodologies** → *Image compression; Virtual reality*; • **Networks** → *Network performance analysis; Application layer protocols; Network mobility; Cloud computing*.

Keywords

Foveated VR Streaming, Reinforcement Learning, VR Cloud Gaming

ACM Reference Format:

Yuk Hang Tsui, Ze Wu, Ahmad Alhilal, Matti Siekkinen, and Pan Hui. 2026. FovRL: Joint Foveation and Quality Control for Immersive VR Streaming Using Reinforcement Learning. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3774904.3792643>

*Co-first author of this work.

†Pan Hui is also affiliated with HKUST and the University of Helsinki, Finland



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792643>

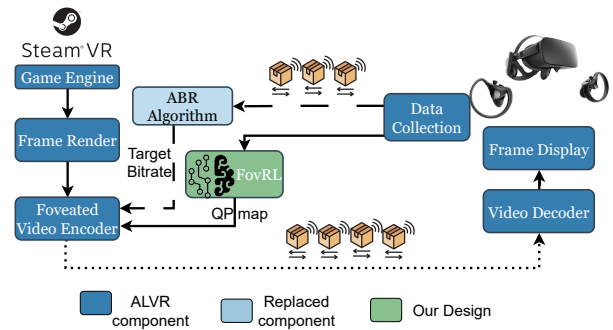


Figure 1: System overview: An RL module for optimizing quality and foveation control produces a QP map as input to the video encoder, integrated into the ALVR platform.

1 Introduction

Virtual reality (VR) cloud gaming offloads intensive rendering to the cloud to enable immersive experiences on lightweight headsets. However, delivering the required high resolution, frame rate, and ultra-low latency is severely challenged by the inherent variability of wireless and mobile networks. The advent of WebXR [15] has enabled instant, browser-based VR, eliminating the need for platform-specific installations. This further intensifies the core streaming challenge, as web browsers are multi-process software that are not optimized for the real-time, deterministic demands of VR streaming.

Two primary approaches exist for seamless streaming. The first integrates adaptive bitrate (ABR) algorithms, which monitor network metrics such as throughput and latency to control the sending bitrate dynamically. This includes integrating the Web Real-Time Communication (WebRTC) framework into VR streaming systems [6, 9] and applying heuristic congestion control (CC)

to application-layer bitrate decisions. While effective for low latency, these methods waste bitrate on peripheral regions, yielding suboptimal perceptual quality. The second approach exploits the non-uniform acuity of the human visual system [25] through foveated video encoding (FVE), which prioritizes quality in the user's focal area. While existing implementations [3, 13] combine FVE with heuristic congestion control, their reliance on rigid, predefined

rules limits adaptation in dynamic networks. Conversely, while reinforcement learning (RL) optimizes network usage in Video On Demand (VoD) [18] and Live streaming [30], its potential for guiding FVE remains unexplored. This leads to our research question: Can RL be applied to FVE to jointly optimize bandwidth utilization and visual quality in dynamic mobile networks?

In this work, we present FovRL, a novel framework that integrates FVE with an RL-based rate controller to optimize bandwidth utilization and visual quality (Figure 1). FovRL meets VR's strict latency and quality demands through three core designs: per-frame control for fine-grained adaptation, online learning for continuous customization to dynamic environments, and the Twin Delayed DDPG (TD3) framework[8] for stable policy updates. To the best of our knowledge, this is the first RL agent expressly designed and trained for immersive VR cloud gaming. We integrate FovRL into an open-source VR gaming platform, where its lightweight design introduces negligible overhead. Our contribution is fourfold:

- **Propose** data-driven adaptive foveation streaming for VR gaming.
- **Design** a frame-level RL inference agent, providing finer control granularity than conventional data-driven streaming, which fits the requirement of VR cloud gaming.
- **Implement** the RL agent with online learning and TD3, yielding a stable, responsive agent that tracks bandwidth shifts without violating strict latency or visual-quality constraints.
- **Evaluate** FovRL over a real deployment after integrating it into an open-source VR gaming platform. FovRL achieves superior bitrate adaptation (bandwidth utilization) and perceptual visual quality.

2 Background and Motivation

VR cloud gaming is constrained by the conflict between ultra-low latency requirements to prevent cybersickness and the high bandwidth needed for immersive visual fidelity. These requirements create an optimization problem that is further complicated by the dynamic nature of bandwidth in real-world networks. This section analyzes how current approaches fail to resolve this tension and motivates our RL-based foveated streaming.

2.1 Limitations of ABR Approaches

Heuristic methods: Video codecs produce an output bitrate using feedback from congestion control algorithms. Google Congestion Control (GCC) [4] and Network-Assisted Dynamic Adaptation (NADA) [32] compute this target using queuing delay and packet loss, while ALVR [1] (an open-source VR platform over WiFi) uses per-frame intervals and bitrates. However, GCC's conservative strategy underutilizes bandwidth, while NADA and ALVR are overly aggressive and cause bitrate instability. Both approaches yield suboptimal latency and visual quality in VR cloud gaming [2]. **Data-driven methods,** particularly RL-based policies like Pensieve [18], have shown significant promise in conventional streaming domains such as Video-on-Demand (VoD) and Live streaming. They optimize quality, smoothness, and rebuffering for higher QoE. However, they fundamentally mismatch with the demands of VR cloud gaming. These methods are designed for a different regime: they typically make decisions on multi-second chunks (e.g., every 2-4 seconds) and operate with latency tolerances of several seconds. In stark contrast, VR cloud gaming requires decisions on a per-frame basis within an end-to-end latency budget of just dozens of milliseconds. This coarse-grained decision-making, combined with

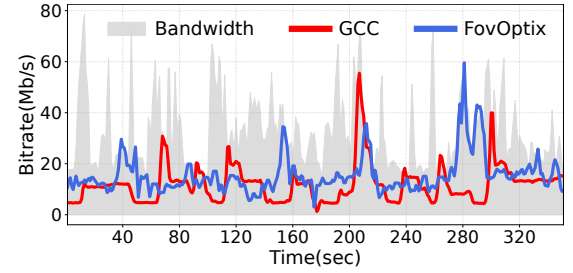


Figure 2: Sending bitrate for GCC and FovOptix versus available bandwidth (gray).

a complete lack of integration with foveated rendering techniques, renders them ineffective for the fine-grained, ultra-low latency requirements of immersive VR.

2.2 Foveation and RL: Potential and Limitations

Foveation enables efficient VR streaming by exploiting non-uniform visual acuity, using Foveated Rendering (FR) [21, 24, 26] to lower peripheral resolution and Foveated Video Encoding (FVE) [3, 13] to apply spatially-varying Quantization Parameter (QP) values. This creates a non-uniform quality map that maintains high fidelity in the foveal region and lower quality in the periphery, where high QP values indicate low quality and vice versa. CGFVE [13] assigns low QP to the fovea and high QP to the periphery. However, its lack of congestion control renders it inefficient under dynamic network conditions. FovOptix [3] implements adaptive foveated encoding by dividing frames into three FoV regions, across which it distributes QPs that adapt to available bandwidth. Current implementations lack intelligent control—they either use network-unaware QP assignment (CGFVE [13]) or inherit limitations from heuristic congestion controllers (FovOptix [3]). While RL excels at network adaptation in traditional streaming [18], existing RL-based ABR methods operate on coarse timescales and lack foveation control. This creates a fundamental gap: no current solution unifies foveation's efficiency with RL's adaptive intelligence for per-frame optimization in VR, leaving their combined potential unexplored.

2.3 Motivation

We benchmark two leading SoTA streaming solutions in VR cloud gaming: the research SoTA FovOptix [3] and the industry-standard GCC [2]. To assess their adaptability to fluctuating network conditions, we measure their sending bitrate under an emulated mobile network within a VR cloud gaming context. The experimental testbed is detailed in section 5. Figure 2 shows that both GCC and FovOptix significantly underutilize available bandwidth. GCC's conservation avoids overshooting but causes persistent undershooting, leading to poor visual quality [2]. While FovOptix uses foveation to optimize perceptual quality, it also undershoots, wasting bandwidth. This inefficiency is detrimental for VR cloud gaming, where high fidelity and low latency are paramount.

Our research aims at building a framework that tightly couples foveated streaming with network adaptation. By mapping an RL agent's actions directly to a foveated QP map, our model dynamically controls video encoding to ensure the bitrate precisely tracks available bandwidth, thereby achieving high fidelity.

3 System Design

This section introduces the architecture of FovRL, a framework for joint latency and perceptual quality optimization in VR cloud gaming. We begin with a system overview and then articulate the central design challenges that guided our methodology.

3.1 System Overview

Our system comprises a cloud gaming server and a lightweight client (see Figure 1). The server renders and encodes frames from the game engine, which are then streamed to the client device (e.g., a VR headset) for decoding and playback. To optimize the trade-off between visual quality and motion-to-photon latency, the client continuously monitors and transmits network statistics and user motion data back to the server. The motion data is fed into the game engine, while FovRL processes the network statistics for adaptive quality control. Unlike existing ABR algorithms providing the target bitrate, FovRL generates a QP map on a frame basis to the foveated video encoder, allowing precise control over the quality distribution across different regions of the frame.

Generating a per-frame QP map with macroblock-level QP assignment introduces complexity to the process of quality allocation. To reduce this overhead, we leverage the foveation parameter C from EyeNexus [27], which provides a simplified model for joint foveation and quality control. The controller manages the standard deviation of the Gaussian curve for assigning the QP value. The higher the C value, the larger the foveation region, and thus is assigned a lower QP value. FovRL's task is streamlined to determine the appropriate C value based on network statistics. The C value is translated into a per-frame QP map that guides the video encoder to perform foveated compression of the rendered frame.

3.2 Design Challenge

The application of RL for foveated streaming faces two key challenges in VR cloud gaming:

(i) **The action (bitrate, foveation) is context-dependent** on the network state and the game's visual and temporal characteristics. This challenge is intensified by the high volatility of mobile networks and the diverse frame complexity across game genres, which varies significantly with visual intricacy and motion dynamics. The core limitation of a pre-trained policy is its inability to adapt to unobservable dynamics; for example, identical network symptoms can stem from different root causes, each demanding a unique response. The static model is forced to apply a single, generalized policy, leading to suboptimal QoE. To address this core limitation, we utilize online learning to tailor the model to the user's specific conditions. During inference, we asynchronously log state-action-reward trajectories and update the policy, enabling real-time adaptation to both network dynamics and game characteristics without interrupting streaming performance. This is critical for VR cloud gaming, where extreme sensitivity to frame size and latency renders pre-trained models ineffective. We validate the necessity of this design through an ablation study in Section 7.1.

(ii) **Instability in Per-Frame Learning.** The frame-level decision-making ensures finer control granularity in highly dynamic environments; however, it leads to highly volatile rewards, as they fluctuate with each frame's encoded size and transient network states. Online learning in this environment amplifies noise, threatening training stability and QoE. Without robust safeguards, this leads to unstable

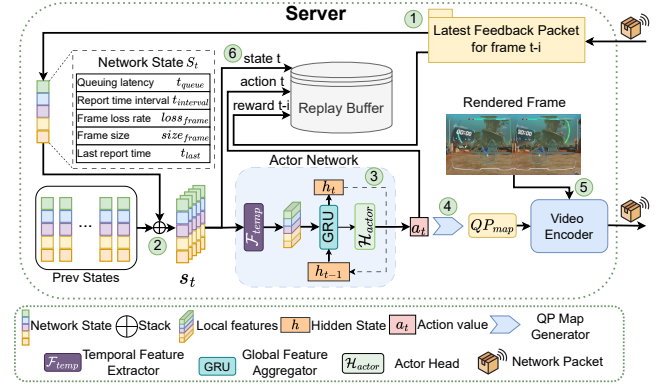


Figure 3: FovRL workflow. (1-2) constructs state S_t by stacking current client data with previous states. (3) This state is fed to the Actor network, which generates an action a_t . (4) a_t is converted into a QP map, which (5) guides the encoding of the rendered frame. (6) Replay Buffer stores state-action (S_t, a_t) and delayed reward r_{t-i} of frame $t-i$ for training.

learning, suboptimal convergence, and poor adaptation to bandwidth changes. To mitigate this, we adopt the TD3 algorithm for its superior noise tolerance and reduced overestimation bias, ensuring stable policy updates. Our implementation is detailed in Section 4.2, with an empirical comparison to DDPG provided in Section 7.2.

4 FovRL: Design and Implementation

In this section, we briefly discuss the design of FovRL, a lightweight RL agent that monitors network conditions and decides the subsequent actions to optimize bandwidth utilization and foveation. We discuss data collection and processing (Section 4.1), including state, action, and reward spaces. Then, we demonstrate the design of the neural network architecture (Section 4.2) that captures both short-term and long-term temporal transmission performance features to assess the current network conditions. Finally, we discuss the implementation of the end-to-end VR cloud gaming system (Section 4.3) after integrating FovRL.

4.1 Data Collection and Processing

At each time step t , the agent forms a state s_t by stacking current and past network states (Figure 3, Steps 1-2) and generates an action a_t (Step 3). After this action is applied to the frame (Steps 4-5), its corresponding reward is only retrieved from a delayed feedback packet (Step 6). The learning objective is to maximize the cumulative reward over the entire gaming session, rather than for any individual frame.

4.1.1 State: The state space provides a comprehensive view of network conditions for adaptive control. It integrates established congestion metrics (queuing delay, report interval, frame loss rate) [4, 5, 10, 11, 29], adapted for per-frame granularity, along with frame size to capture encoding variability. To model temporal dynamics, we form the final state by concatenating the current observation with $n-1$ previous states, providing the necessary context to discern trends. The feature selection rationale is detailed below:

Queuing latency (t_{queue}) is a fundamental congestion indicator. A gradual increase suggests that the sending bitrate may exceed

available bandwidth, signaling impending congestion. We estimate t_{queue} as the difference between the inter-packet arrival and sending intervals for consecutive frames:

$$t_{queue} = (t_{arr}^i - t_{arr}^{i-1}) - (t_{send}^i - t_{send}^{i-1}) \quad (1)$$

where t_{arr}^i and t_{send}^i denote the arrival and sending times of the last packet of frame i , respectively.

Report time interval ($t_{interval}$) measures the time between consecutive client feedback packets. An increase in this interval indicates network delays from queuing or bottlenecks, providing a complementary measure of congestion. A large $t_{interval}$ indicates stale network data in the feedback packet, thus useless data that misrepresents the current conditions.

Frame loss rate ($loss_{frame}$) is calculated over a 500 ms sliding window. Each video frame is divided into multiple UDP packets for transmission, and the loss of any packet renders the entire frame undecodable. A frame loss rate of 0 guarantees smooth experiences.

Frame size ($size_{frame}$) is the encoded byte size of the frame associated with the feedback. It directly indicates the sent volume and, when combined with other metrics, aids in bandwidth estimation. Monitoring frame size helps understand how different data volumes affect congestion, providing key insights for network adaptation.

Last report time (t_{last}) measures the time difference between the network state retrieval and the feedback arrival. This metric is crucial for detecting scenarios where network congestion causes feedback packets to be lost, leaving the model with stale state information. It enables the model to adapt its behavior accordingly when operating on outdated data.

After the previous frame is encoded, t_{queue} , $t_{interval}$, p_{loss} , and $size_{packet}$ are retrieved from the latest feedback packets, and t_{last} is computed to form the network state. This state is concatenated with the previous $n - 1$ states to form the input state $s \in \mathbb{R}^{n \times 5}$.

4.1.2 Action: Directly predicting a unique QP for each macroblock within the frame is computationally intractable due to the resulting high-dimensional action space. To overcome this, we simplify the agent's task to output a single, continuous value: the foveation controller C . Following the foveation model proposed by EyeNexus [27], this value C is used as a parameter in a deterministic function that generates the final QP map for the encoder. The QP map follows a Gaussian distribution of quality centered on the user's gaze point:

$$QP_{map}[i, j] = QP_{max} - QO_{max} \times \exp\left(-\frac{Dist(i, j)^2}{2C^2}\right) \quad (2)$$

QP_{max} is the maximum QP value (lowest quality), QO_{max} is the maximum QP offset, and $Dist(i, j)$ is the Euclidean distance from a macroblock to the gaze point. The agent's action, C , directly controls the standard deviation of this Gaussian function, which determines the size of the foveal region. A larger C creates a wider foveal area with lower QP values, thus increasing both visual quality and bitrate. This design reduces the agent's task to learning a single parameter that governs the quality-bandwidth trade-off. Formally, the action at each step is defined as $a_t = C$, where $a_t \in \mathbb{R}$.

4.1.3 Reward: This is designed to reflect the user's QoE, which balances visual quality with latency. We refine these metrics to reflect QoE. Network latency (t_{net}) is transformed into a refined

latency (l_{re}) using a polynomial to model the exponentially increasing penalty of high delays. The action C value is translated into a refined visual quality score (vq_{re}) to account for the non-linear relationship between the action and its perceived visual effect.

$$\begin{aligned} l_{re} &= (t_{net} * t_{scale})^{t_{deg}} \\ ew_{map}[i, j] &= \frac{1}{2\pi\sigma^2} \exp\left(-\frac{Distance(i, j)^2}{2\sigma^2}\right) \\ vq_{re} &= \sum_{(i, j) \in QP_{map}} (ew_{map}[i, j] * (QP_{max} - QP_{map}[i, j])) \\ \tilde{vq}_{re} &= \frac{2vq_{re}}{\max(vq_{re})} - 1 \end{aligned} \quad (3)$$

In our implementation, we set $t_{scale} = 0.05$ and $t_{deg} = 1.5$ based on empirical tuning. The refined visual quality, vq_{re} , is calculated by weighting macroblock quality with the gaze-centric ew_{map} (using $\sigma = 20$) and then normalizing the result to $(-1, 1]$ as $\tilde{vq}_{re}$. The total reward is formed from four components: a visual quality reward (r_{vq}), a latency penalty (r_{lat}), a quality reduction encourager (r_{qre}), and a smoothness penalty (r_{sm}).

$$\begin{aligned} r_{vq} &= \frac{\tilde{vq}_{re}}{l_{re}}, & r_{lat} &= l_{re} \\ r_{qre} &= \frac{l_{re}}{\tilde{vq}_{re} + 1}, & r_{sm} &= \max(0, |a_t - a_{t-1}|) - thres_{sm} \\ reward &= \alpha r_{vq} - \beta r_{lat} + \gamma r_{qre} - \delta r_{sm} \end{aligned} \quad (4)$$

The visual quality (VQ) reward, r_{vq} , is designed to reward high visual quality ($\tilde{vq}_{re}$) relative to its latency cost (l_{re}). r_{vq} is assigned a weight α in the total reward to balance high VQ and low latency. **The latency penalty**, r_{lat} , is applied to the refined latency (l_{re}), punishing any increase in network latency.

The Quality reduction encourager, r_{qre} , incentivizes the agent to strategically reduce visual quality when latency becomes excessively high, thereby helping to alleviate network congestion.

The smooth penalty, r_{sm} , penalizes large changes between consecutive actions (a_t and a_{t-1}). By incorporating a threshold ($thres_{sm} = 2$), it creates a dead zone that allows for minor adjustments without penalty, thereby promoting temporal smoothness and preventing jarring quality fluctuations.

The reward weights (Equation 4) are set empirically to $\alpha = 0.9$ (VQ), $\beta = 0.4$ (latency), $\gamma = 0.25$ (trade-off), and $\delta = 0.02$ (smoothness). They balance the scale of each reward component to prioritize high VQ while penalizing excessive latency strongly. The low smoothness weight is intended for agile policy adaptation. Notably, the model converges reliably even when these weights are adjusted to shift policy priorities.

4.2 Neural Network Design and Training

FovRL QoE through continuous online adaptation of its control policy. As motivated in Section 3.2, the dynamic nature of network environments necessitates an online RL approach for continuous learning and fine-tuning. This section details two FovRL's key components: the neural network architecture and the advanced training framework employed to ensure stable and efficient convergence.

Actor π_ϕ - Critic Q_θ NN architecture. To effectively navigate volatile network conditions, our actor-critic model employs a shared backbone to extract features from the temporal network state. This

common representation is then fed into separate actor and critic heads with independently trained weights, allowing the model to learn nuanced policies for managing dynamic network behavior. First, 1-D Convolutional Neural Networks (CNNs), $\mathcal{F}_{temp}$, process the input network statistics (s_i) to capture short-term patterns. Its convolutional filters identify localized motifs, such as packet bursts or latency spikes, to produce a condensed feature vector f_i .

$$f_i = \mathcal{F}_{temp}(s_i) \quad (5)$$

To model long-term dependencies, this feature vector f_i is then passed to a Gated Recurrent Unit (GRU). The GRU sequentially integrates these features with its hidden state (h_{i-1}), which summarizes historical context, to produce a comprehensive state representation g_i that captures the broader network trend.

$$g_i, h_i = GRU(f_i, h_{i-1}) \quad (6)$$

Finally, g_i is fed into the actor ($\mathcal{H}_{actor}$) and critic ($\mathcal{H}_{critic}$) heads, both of which are implemented as dense neural networks (DNNs). The actor maps the state g_i to a control action a_i , while the critic evaluates this action by processing both the state g_i and the action a_i to estimate the Q-value, Q_i . This separation allows the shared backbone to learn a rich representation while the heads specialize in their respective tasks of policy creation and value estimation.

$$\begin{aligned} a_i &= \mathcal{H}_{actor}(g_i) \\ Q_i &= \mathcal{H}_{critic}(g_i, a_i) \end{aligned} \quad (7)$$

Online Learning: To supply the GRU architecture with the necessary temporal context, the replay buffer stores experience as overlapping sequences of m consecutive transition tuples (state, action, reward). When training on a sampled sequence, the initial windows are used only to warm up the GRU's hidden state and are excluded from backpropagation. This ensures gradients are computed from states with sufficient history. The inference agent's weights are updated by synchronizing them with the weights from the newly trained model.

Stability: As stated in the challenges (Section 3.2), frame-level decision-making leads to highly variable rewards, causing training instability and promoting the overestimation of Q-values. Therefore, we employ the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm to train actor π_θ and critic Q_ϕ networks. TD3 enhances stability through 1) Clipped Double Q-learning to provide a conservative value estimate to counteract overestimation, 2) Target policy smoothing to regularize the value function against sharp, erroneous peaks, and 3) Delayed policy updates to ensure policy improvements are based on well-converged value estimates by decoupling the actor and critic updates.

Algorithm 1 details the training process. After sampling a batch of transitions, the target action a' is computed via target policy $\pi_{\theta_{targ}}$ smoothing by adding clipped noise to the target actor's output, which regularizes the value function against sharp, exploitable peaks. Next, the learning target y is calculated using Clipped Double Q-learning, taking the minimum value from the twin target critic networks to form a conservative estimate that mitigates Q-value overestimation. This target is then used to update both critic networks via gradient descent. In accordance with delayed policy updates, the actor and target networks are updated less frequently than the critics, allowing the value estimate to stabilize before it

guides the policy. Following the actor update, the target networks are updated using a soft Polyak average. A crucial detail for our GRU-based architecture is that when computing losses, the initial few time steps of each sequence are disregarded. This "warm-up" period ensures the GRU's hidden state is conditioned on temporal information before contributing to the gradient calculation.

Algorithm 1 Twin Delayed Deep Deterministic Policy Gradient

Input: m consecutive window of observation: $D \in \langle S, \mathcal{A}, \mathcal{R} \rangle$

repeat

Randomly sample a batch of transitions,
 $B = \{(s, a, r, s')\}$ from D
 Compute target action: $\triangleright$ target policy smoothing
 $a'(s') = clip(\pi_{\theta_{targ}}(s') + clip(\epsilon, -c, c), a_{low}, a_{high})$
 Compute target: $\triangleright$ Clipped double Q learning
 $y \leftarrow r + \gamma \cdot \min_{i=1,2} Q_{\phi_{targ,i}}(s', a'(s'))$
 Update twin Q function: $\triangleright$ Dropped first few window
 $\nabla_{\phi} \frac{1}{|B|} \sum_{(s,a,r,s') \in B} (Q_{\phi_i}(s, a) - y)^2$ for $i = 1, 2$

if $update_count \% 2 = 0$ **then**

Update actor network: $\triangleright$ Delayed Policy Updates
 $\nabla_{\theta} \frac{1}{|B|} \sum_{s \in B} Q_{\phi_1}(s, \pi_{\theta}(s))$ $\triangleright$ Dropped first few window
 Update Target network:
 $\phi_{targ,i} \leftarrow \rho \phi_{targ,i} + (1 - \rho) \phi_i$ for $i = 1, 2$
 $\theta_{targ} \leftarrow \rho \theta_{targ} + (1 - \rho) \theta$

$update_count += 1$

until *end of gameplay session*;

4.3 Implementation

Our system comprises server and client components (see Figure 1) communicating over the internet. We implement FovRL within the ALVR open-source framework [1], configured for a framerate of 72 Hz and a resolution of 3712×1920 . In our implementation, FovRL replaces the original ABR method and directly generates the QP map for the video encoder to control the final encoded frame size. A critical challenge in this integration is ensuring the agent's operations do not introduce latency and stall the real-time streaming process. Therefore, we decouple the agent's inference and training processes from the encoding and transmission processes through a multi-threaded architecture.

FovRL. We implement the RL agent and training logic (see Section 4.2) using C++ and the LibTorch library. To prevent blocking the real-time streaming pipeline, training and inference run in separate threads. Inference is triggered after frame transmission, ensuring no latency impact. we incorporate techniques to ensure stable online learning: (1) initialization from a pre-trained checkpoint conducted on the testbed (section 5) to mitigate early instability; (2) a dual-model architecture to decouple learning and inference, resolving the operational conflict between them in a single stateful model; (3) Periodic weight update of the inference model with the latest learned weights; and (4) after weight synchronization, re-initializing the inference model's GRU state with recent observations to preserve temporal context and prevent performance degradation.

Foveated Video Encoder. FovRL's action is translated into a QP offset map, which configures the video encoder's state to implement foveated video encoding. We utilize NVIDIA's Video Codec SDK [20] to perform H.264 hardware-accelerated video encoding (NVENC). To ensure low latency, the encoder is set up with an infinite GOP length and a speed-optimized preset. Foveated video encoding is achieved by activating the Constant QP rate control

mode in NVENC. In const QP mode, the QP map of each frame is formed by adding the base QP to the QP offset map. In our implementation, we configure the base QP ($QP_{const} = 15$) and pass the QP offset map based on the model action for every frame.

5 Experiment Setup

This details the experimental testbed to emulate real-world VR cloud gaming, network traces used for emulation, and baselines.

5.1 Experimental Testbed

We design a physical testbed to emulate cloud gaming under variable network conditions. We set up a Linux Computer with WiFi and Ethernet interfaces (Ubuntu 22.04.4 LTS, Intel(R) Core(TM) i9-14900KF @ 6.0 GHZ) to act as a PC router. The PC router connects to the gaming server (Windows 11 24H2, Intel(R) Core(TM) i9-13900K @ 5.8GHz, NVIDIA GeForce RTX 4080 GPU, Driver Version 581.15) via the Ethernet interface. It also connects to an Oculus Quest 3[19](Qualcomm Snapdragon XR2 Gen 2, Octa-core Kryo (1 x 3.19 GHz, 4 x 2.8 GHz, 3 x 2.0 GHz), Adreno 740) via the WiFi interface. The PC router enables IP and packet forwarding to connect the gaming server to the Oculus Quest 3. It regulates bandwidth between the server and client using Linux Traffic Control (tc) [12] to emulate diverse network conditions from network traces.

5.2 Network Traces and Baselines

Our network traces help to replicate real network conditions between VR headsets and cloud server. We leverage a dataset of 5G mobile network traces collected from a major Irish operator [22], generated under three traffic conditions: continuous large file downloads, and video streaming from both Netflix and Amazon Prime.

Our baselines cover heuristic rate control methods with foveation (FovOptix and ALVR) and without foveation (GCC), providing comprehensive benchmarks. ALVR is the open-source VR gaming over WiFi [1]. Google Congestion Control (GCC) is the industry standard rate control for real-time media [4] that is integrated into VR cloud Gaming and evaluated extensively in [2]. FovOptix[3] is the SoTA bandwidth-adaptive FVE. We compare them against FovRL, which leverages an RL-based rate control policy to optimize foveation. We pretrain FovRL on a Wi-Fi network (≈ 20 Mbps) to ensure initial stability, different from the evaluation network (mobile network).

5.3 Evaluation Setup

There are two major setups for server-rendered VR gaming: 1) **Local rendering**: VR headsets are connected to a high-end PC via WiFi, which provides sufficient bandwidth for video streaming. We set up a PC router to connect the VR headset to the high-end PC and limit the bandwidth to 80 Mbps to emulate a concurrent WiFi sharing of multiple users; and 2) **Cloud rendering**: VR headsets are connected to a cloud server via mobile networks, which exhibit variations in available bandwidth. The PC router regulates the bandwidth according to network traces.

6 Performance Evaluation

For a fair evaluation, we select widely played VR games from three distinct genres: Action/Shooting (Star Legacy), Rhythm/Music (Beat Saber), and Casual/Exploration (VR Puzzle). These genres are representative of the broader VR gaming landscape [7, 28]. For each game, we collect approximately 70k frames (10 minutes) per baseline, resulting in 70k data points for performance comparison. We then assess the performance based on bitrate adaptation, visual quality (VQ), and Motion-to-Photon (MTP) latency.

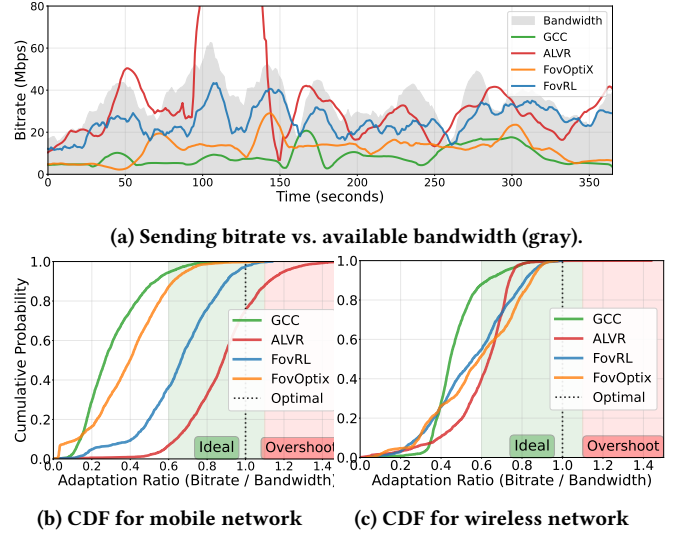


Figure 4: Sending bitrate and CDF of bitrate adaptation ratio (sending bitrate / available bandwidth) across all baselines. The ideal range is within the green band [0.6, 1.1], while the overshoot range is within the red band (> 1.1).

6.1 Bitrate Adaptation

Figure 4a compares bitrate adaptation under mobile networks. While GCC, FovOptix, and FovRL reduce overshoot (unlike ALVR), only FovRL closely tracks bandwidth with minimal undershoot. GCC and FovOptix exhibit frequent, substantial undershoot, resulting in lower bandwidth utilization and thus lower visual quality (subsection 6.3). This conservative approach also explains their lower latency (subsection 6.2) compared to FovRL. This is evident in Figure 4b and Figure 4c, which present the bitrate adaptation ratio, defined as the sending bitrate to available bandwidth ratio.

Mobile Networks (Figure 4b). FovRL utilizes 60% to 110% (ideal range) of the available bandwidth for 57% of the gameplay, with only 1% of the gameplay exceeding 110% (overshoot range). GCC and FovOptix exhibit almost no bandwidth overshooting; however, 94% and 88% of their gameplay, respectively, operate below 60% bandwidth utilization, underscoring the limitations of heuristic-based control under highly dynamic network conditions. Although ALVR attains high bandwidth utilization—92% of gameplay above 60% usage—it frequently overshoots, with 12% of the time exceeding 110% of available bandwidth, leading to unacceptable latency.

WiFi Networks (Figure 4c). Under stable high-bandwidth conditions, ALVR operates within the ideal utilization range [60%, 110%] for 60% of gameplay, aligning with its high-throughput design. FovRL (44%) and FovOptix (47%) show moderate efficiency, whereas GCC remains ineffective, achieving ideal utilization for only 12% of the gameplay. Moreover, ALVR and GCC exhibit bandwidth utilization exceeding 80% for less than 2%. In contrast, FovRL and FovOptix utilize the bandwidth more efficiently, exceeding 80% for 12.2% and 16.9% of gameplay, respectively. This demonstrates their ability to leverage high bandwidth for superior visual quality, explaining their higher VQ performance in Section 6.3.

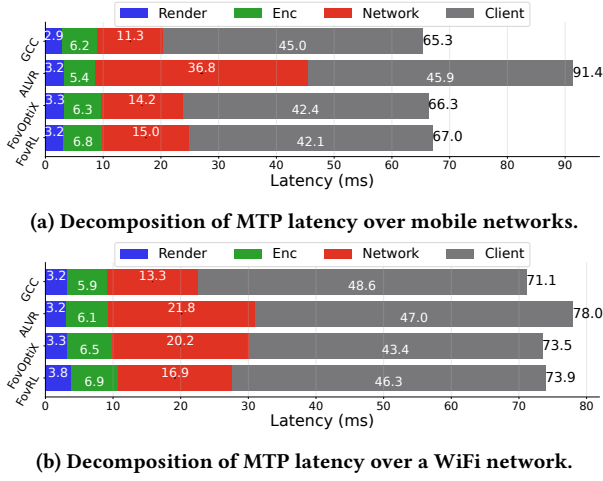


Figure 5: Decomposition of MTP latency into server (Render and Encode), network, and client latency. FovRL adds marginal encoding overhead.

Overall, the results confirm the inferior bitrate adaptability of SoTA methods in handling dynamic bandwidth fluctuations in mobile networks. FovRL demonstrates significant improvements in bandwidth adaptability and streaming efficiency, resulting in a lower percentage of bitrate overshoot and undershoot.

6.2 Network and total latency

Mobile Network. Figure 5a shows the latency decompositions over emulation of 5G mobile networks. GCC, FovOptix, and FovRL exhibit comparable total latencies, with an average of 65.3 ms, 66.4 ms, and 67.1 ms, respectively. The major difference resides in the network latency. GCC achieves the lowest latency (11.3 ms), followed by FovOptix (14.2 ms) and FovRL (15 ms). GCC’s conservative control strategy prioritizes low latency over bandwidth utilization, resulting in a lower sending bitrate (see Section 6.1). ALVR has a significantly higher total latency (91.4ms), with 36.8 ms of network latency, which is attributed to the high overshooting percentage that causes increased latency during the overshooting period. The encoding latency of FovRL is higher than that of the other two by around 0.7 ms due to model training and inference, which introduces a computational burden. The average inference time of the model is 0.805 ms, and the average training time for one epoch is 3.86 sec (5.24 ± 0.33 sec with actor update and 2.58 ± 0.17 sec without actor update). This demonstrates that the separation of the training model and the inference model, as presented in Section 4.3, is necessary to avoid introducing extra latency to the pipeline.

Wireless Network. As shown in Figure 5b, FovRL’s end-to-end latency (73.9 ms) is highly competitive, closely matching FovOptix (73.5 ms) and remaining in the same range as GCC (71.1 ms). The increase mainly lies in network latency, which is possibly due to a higher sending bitrate being used in the Wireless network, leading to slightly higher transmission latency. This causes higher bitrate variation and makes congestion easier to occur.

Although our FovRL introduces additional workload for online training and inference, it achieves a comparable MTP latency to

Table 1: Visual Quality Comparison. Higher is better for EW-PSNR (dB) and EWSSIM; lower is better for LPIPS.

(a) VQ comparison under mobile network

Baseline	EWPSNR (dB) ↑	EWSSIM ↑	LPIPS ↓
GCC	31.04 ± 2.34	0.69 ± 0.05	0.36 ± 0.09
ALVR	31.46 ± 1.65	0.68 ± 0.05	0.30 ± 0.08
FovOptix	31.98 ± 0.92	0.82 ± 0.06	0.27 ± 0.06
FovRL	33.76 ± 2.39	0.83 ± 0.04	0.22 ± 0.06

(b) VQ comparison under WiFi network

Baseline	EWPSNR (dB) ↑	EWSSIM ↑	LPIPS ↓
GCC	31.41 ± 1.82	0.67 ± 0.06	0.29 ± 0.08
ALVR	31.04 ± 2.12	0.63 ± 0.11	0.31 ± 0.11
FovOptix	33.5 ± 1.48	0.86 ± 0.09	0.20 ± 0.05
FovRL	33.87 ± 2.07	0.84 ± 0.05	0.19 ± 0.06

baselines under various network conditions. This ensures its potential to provide seamless streaming and immersive VR gaming.

6.3 Visual Quality (VQ)

We assess the VQ using EWPSNR [16], EWSSIM [23], and LPIPS [31]. EWPSNR and EWSSIM are designed for evaluating foveated frames [14, 27]. They leverage an eye-weight map, which assigns weights to image pixels according to the gaze location, with a gradual decay. LPIPS evaluates the perceptual similarity, focusing more on perceptual quality than pixel-wise accuracy. Table 1 compares the VQ for the baselines over mobile and WiFi networks. Over mobile networks, FovRL achieves the highest VQ across the three metrics—attributable to superior bandwidth utilization. GCC and ALVR exhibit consistently lower scores. Compared to FovOptix, FovRL demonstrates a wider gap in EWPSNR and LPIPS, while achieving similar performance on EWSSIM. This suggests that FovOptix effectively preserves the structural information in the foveal region during compression but sacrifices finer details, texture fidelity, and perceptual consistency, which impacts overall image quality as perceived by humans. In a wireless network environment, FovRL exhibits a modest improvement in visual quality while retaining the best EWPSNR and LPIPS. FovOptix exhibits a substantial quality increase, indicating strong performance under stable, ample bandwidth conditions, but weaker adaptation to dynamic changes. GCC achieves slight overall gains, whereas ALVR shows no improvement relative to the mobile setting. Considering latency, ALVR’s rate control overestimates available bandwidth in mobile networks, yielding higher visual quality at the cost of elevated latency.

Overall, FovRL demonstrates a clear advantage in visual quality, attributed to significant improvements in bandwidth utilization under highly dynamic network conditions (Section 6.1).

7 Ablation Study

We further investigate the importance of online learning and the significance of the TD3 algorithm. As such, we conduct additional experiments over the same testbed described in Section 5.1.

7.1 Importance of Online Learning

We benchmark against FovRL(frozen), a variant with an identical architecture but a fixed, offline pre-trained policy. To ensure a fair comparison, FovRL(frozen) was pre-trained on the exact same

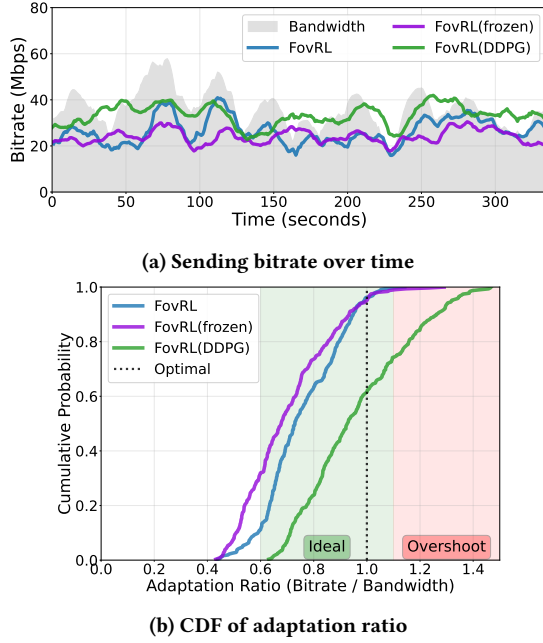


Figure 6: Sending Bitrate and Cumulative probability (CDF) of the bitrate adaptation ratio across FovRL variants.

game and network traces used for evaluation. Despite the ideal pre-training, FovRL(frozen) shows significantly lower responsiveness to network dynamics. Figure 6 shows that FovRL(frozen) bitrate adaptability is inferior to FovRL. With a sending bitrate of just 65% versus FovRL’s 88%, FovRL(frozen) fails to respond adequately to bandwidth fluctuations, despite similar overall performance. Notably, online learning introduces a negligible server latency overhead of 0.37 ms per inference. Conversely, the lack of FovRL(frozen) adaptability causes frequent overshoots, resulting in a 0.23 ms average increase in network latency. These results confirm that real-time online learning is indispensable for maximizing efficiency and quality control, outperforming even a perfectly pre-trained model.

7.2 Significance of TD3

To isolate the stability improvements of the TD3 algorithm, we compare it directly to its predecessor, FovRL(DDPG) [17], holding the network architecture and MDP formulation (state, action, reward) constant. The results reveal that DDPG’s aggressive nature is detrimental in volatile network conditions. As shown in Figure 6a, FovRL(DDPG) consistently overshoots the available bandwidth, behaving aggressively on minor dips while only reacting to major drops. The CDF in Figure 6b quantifies this instability, where the agent spends a staggering 25% of its gameplay in overshoot. This aggressive behavior incurs a significant penalty, increasing network latency by 4.67 ms compared to FovRL(TD3). Its architectural improvements—specifically twin critics and target policy smoothing—curb the overestimation bias that drives DDPG’s aggression. This results in fewer overshoots and more precise action, leading to superior real-time control and substantially lower latency.

This ablation study validates our core design decisions against the fundamental challenges identified in Section 3.2. First, online

learning proves indispensable for overcoming context-dependent volatility, as demonstrated by the failure of static models even with ideal pre-training. Second, TD3’s stabilized learning is essential for reliable per-frame control, successfully mitigating the noise sensitivity and estimation bias that plague DDPG implementations.

8 Discussion and Limitations

Our findings reveal that FovRL addresses the bandwidth adaptation challenge. It delivers substantially higher visual quality than existing methods on mobile networks, characterized by markedly lower undershoot and negligible overshoot compared to GCC and FovOptix. FovRL also achieves superior visual quality over stable Wi-Fi. Crucially, FovRL achieves these gains by introducing a negligible latency overhead. Unlike heuristic methods like GCC and FovOptix, FovRL employs an online learning using RL, which is a novel intelligent quantization and rate control in a VR cloud gaming context. The RL agent learns to dynamically allocate quality levels across the FoV while simultaneously adapting the output bitrate to real-time bandwidth estimates. While the strict latency demands of VR have hindered the application of RL, FovRL is lightweight, performing per-frame inference with minimal overhead.

Our ablation study confirms that the computational cost is modest relative to the substantial improvements in bandwidth adaptation and visual quality. This overhead is readily absorbed when deploying on cloud servers with ample resources.

Limitations. A key limitation is our reliance on objective metrics—latency and perceptual VQ—as standard surrogates for QoE. While improvements in our reward design demonstrate gains in these metrics, a user study or subjective MOS evaluation is necessary to confirm that they translate into improvements in actual player experience, such as immersion, comfort, and sickness reduction. A further limitation is the empirical basis for our reward function. While the weights were tuned to prioritize bitrate adaptability, the work lacks a sensitivity analysis of the weights to show how performance varies with different configurations, which would strengthen the validation of our chosen values. We plan to conduct a sensitivity analysis and formal theoretical analysis to justify the choices of state and weight values in future works.

9 Conclusion

This paper presented FovRL, a novel framework that brings reinforcement learning (RL) to foveated VR streaming. FovRL combines online learning with Twin Delayed Deep Deterministic Policy Gradient (TD3) to achieve stable, per-frame control of foveation parameters. This fine-grained control allows FovRL to adapt to bandwidth dynamics in real-time by intelligently allocating bitrate across foveal regions, meeting the strict latency constraints of VR. We integrated FovRL into an open-source VR gaming platform and evaluated it over a real-world gaming setup. Our findings revealed that FovRL achieves superior bandwidth utilization and perceptual visual quality. This work demonstrated the potential of applying modern RL techniques to optimize complex, real-time foveation and quality in latency-sensitive cloud-rendered VR applications.

Acknowledgments

This work was supported in part by the National Key Research and Development Program of China under Grant 2024YFC3307602, the Guangdong Provincial Talent Program, Grant No. 2023JC10X009, and the funding from the HSBC project, Grant No. L0562.

References

- [1] 2024. Air Light VR (ALVR). <https://github.com/alvr-org/ALVR> <https://github.com/alvr-org/ALVR>.
- [2] Ahmad Alhilal, Ze Wu, Teemu Kämäräinen, Tristan Braud, and Matti Siekkinen. 2025. Congestion Control for VR Cloud Gaming: Integration and Comparison in Real VR Gaming Environment. doi:10.1145/3746027.3755439
- [3] Ahmad Alhilal, Ze Wu, Yuk Hang Tsui, and Pan Hui. 2024. FovOptix: Human Vision-Compatible Video Encoding and Adaptive Streaming in VR Cloud Gaming. In *Proceedings of the 15th ACM Multimedia Systems Conference (Bari, Italy) (MMSys '24)*. Association for Computing Machinery, New York, NY, USA, 67–77. doi:10.1145/3625468.3647612
- [4] Gaetano Carlucci, Luca De Cicco, Stefan Holmer, and Saverio Mascolo. 2016. Analysis and Design of the Google Congestion Control for Web Real-Time Communication (WebRTC). In *Proceedings of the 7th International Conference on Multimedia Systems (Klagenfurt, Austria) (MMSys '16)*. Association for Computing Machinery, New York, NY, USA, Article 13, 12 pages. doi:10.1145/2910017.2910605
- [5] Gaetano Carlucci, Luca De Cicco, Stefan Holmer, and Saverio Mascolo. 2017. Congestion Control for Web Real-Time Communication. *IEEE/ACM Transactions on Networking* 25, 5 (2017), 2629–2642. doi:10.1109/TNET.2017.2703615
- [6] Sylvie Dijkstra-Soudarissanane, Karim El Assal, Simon Gunkel, Frank ter Haar, Rick Hindriks, Jan Willem Kleinrouweler, and Omar Niamut. 2019. Multi-sensor capture and network processing for virtual reality conferencing. In *Proceedings of the 10th ACM Multimedia Systems Conference (Amherst, Massachusetts) (MMSys '19)*. Association for Computing Machinery, New York, NY, USA, 316–319. doi:10.1145/3304109.3323838
- [7] Maxwell Foxman, Alex P. Leith, David Beyea, Brian Klebig, Vivian Hsueh Hua Chen, and Rabindra Ratan. 2020. Virtual Reality Genres: Comparing Preferences in Immersive Experiences and Games. In *Extended Abstracts of the 2020 Annual Symposium on Computer-Human Interaction in Play (Virtual Event, Canada) (CHI PLAY '20)*. Association for Computing Machinery, New York, NY, USA, 237–241. doi:10.1145/3383668.3419881
- [8] Scott Fujimoto, Herke van Hoof, and David Meger. 2018. Addressing Function Approximation Error in Actor-Critic Methods. arXiv:1802.09477 [cs.AI] <https://arxiv.org/abs/1802.09477>
- [9] Simon N. B. Gunkel, Rick Hindriks, Karim M. El Assal, Hans M. Stokking, Sylvie Dijkstra-Soudarissanane, Frank ter Haar, and Omar Niamut. 2021. VRComm: an end-to-end web system for real-time photorealistic social VR communication. In *Proceedings of the 12th ACM Multimedia Systems Conference (Istanbul, Turkey) (MMSys '21)*. Association for Computing Machinery, New York, NY, USA, 65–79. doi:10.1145/3458305.3459595
- [10] David A. Hayes and Grenville Armitage. 2011. Revisiting TCP congestion control using delay gradients. In *Proceedings of the 10th International IFIP TC 6 Conference on Networking - Volume Part II (Valencia, Spain) (NETWORKING '11)*. Springer-Verlag, Berlin, Heidelberg, 328–341.
- [11] Tianchi Huang, Rui-Xiao Zhang, Chao Zhou, and Lifeng Sun. 2018. QARC: Video Quality Aware Rate Control for Real-Time Video Streaming based on Deep Reinforcement Learning. In *Proceedings of the 26th ACM International Conference on Multimedia (Seoul, Republic of Korea) (MM '18)*. Association for Computing Machinery, New York, NY, USA, 1208–1216. doi:10.1145/3240508.3240545
- [12] Bert Hubert. 2001. <https://man7.org/linux/man-pages/man8/tc.8.html>
- [13] Gazi Karam Illahi, Thomas Van Gemert, Matti Siekkinen, Enrico Masala, Antti Oulasvirta, and Antti Ylä-Jääski. 2020. Cloud Gaming with Foveated Video Encoding. *ACM Trans. Multimedia Comput. Commun. Appl.* 16, 1, Article 7 (Feb. 2020), 24 pages. doi:10.1145/3369110
- [14] Gazi Karam Illahi, Thomas Van Gemert, Matti Siekkinen, Enrico Masala, Antti Oulasvirta, and Antti Ylä-Jääski. 2020. Cloud Gaming with Foveated Video Encoding. *ACM Trans. Multimedia Comput. Commun. Appl.* 16, 1, Article 7 (Feb. 2020), 24 pages. doi:10.1145/3369110
- [15] Immersive Web Community Group. 2025. Immersive Web Developer Home: The WebXR Device API. <https://immersiveweb.dev/> Accessed: 2025-10-05.
- [16] Zhicheng Li, Shiyin Qin, and Laurent Itti. 2011. Visual attention guided bit allocation in video compression. *Image Vision Comput.* 29, 1 (Jan. 2011), 1–14. doi:10.1016/j.imavis.2010.07.001
- [17] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2015. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971* (2015).
- [18] Hongzi Mao, Ravi Netravali, and Mohammad Alizadeh. 2017. Neural Adaptive Video Streaming with Pensieve. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication (Los Angeles, CA, USA) (SIGCOMM '17)*. Association for Computing Machinery, New York, NY, USA, 197–210. doi:10.1145/3098822.3098843
- [19] Meta Platforms, Inc. 2025. Meta Quest Official Website. <https://www.meta.com/quest/>. Accessed: 2025-08-21.
- [20] NVIDIA. 2023. Video Codec SDK. <https://developer.nvidia.com/video-codec-sdk> Accessed: 2024-11-30.
- [21] Anjul Patney, Marco Salvi, Joohwan Kim, Anton Kaplanyan, Chris Wyman, Nir Benty, David Luebke, and Aaron Lefohn. 2016. Towards foveated rendering for gaze-tracked virtual reality. *ACM Trans. Graph.* 35, 6, Article 179 (Dec. 2016), 12 pages. doi:10.1145/2980179.2980246
- [22] Darijo Raca, Dylan Leahy, Cormac J. Sreenan, and Jason J. Quinlan. 2020. Beyond throughput, the next generation: a 5G dataset with channel and context metrics. In *Proceedings of the 11th ACM Multimedia Systems Conference (Istanbul, Turkey) (MMSys '20)*. Association for Computing Machinery, New York, NY, USA, 303–308. doi:10.1145/3339825.3394938
- [23] Snjezana Rimac-Drlje, Goran Martinović, and Branka Zovko-Cihlar. 2011. Foveation-based content Adaptive Structural Similarity index. In *2011 18th International Conference on Systems, Signals and Image Processing*. 1–4.
- [24] Tim Rolf, Susanne Schmidt, Ke Li, Frank Steinicke, and Simone Frintrop. 2023. VRS-NeRF: Accelerating Neural Radiance Field Rendering with Variable Rate Shading. In *2023 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. 243–252. doi:10.1109/ISMAR59233.2023.00039
- [25] Louis D. Silverstein. 1996. Foundations of Vision, by Brian A. Wandell, Sinauer Associates, Inc., Sunderland, MA, 1995. xvi + 476 pp., hardcover \$49.95. *Color Research & Application* 21, 2 (1996), 142–144. arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/col.5080210213 doi:10.1002/col.5080210213
- [26] David R. Walton, Rafael Kuffner Dos Anjos, Sebastian Fris-ton, David Swapp, Kaan Aksit, Anthony Steed, and Tobias Ritschel. 2021. Beyond blur: real-time ventral metamers for foveated rendering. *ACM Trans. Graph.* 40, 4, Article 48 (July 2021), 14 pages. doi:10.1145/3450626.3459943
- [27] Ze Wu, Ahmad Alhilal, Yuk Hang Tsui, Matti Siekkinen, and Pan Hui. 2025. EyeNexus: Adaptive Gaze-Driven Quality and Bitrate Streaming for Seamless VR Cloud Gaming Experiences. *Proc. ACM Netw.* 3, CoNEXT4, Article 42 (Nov. 2025), 25 pages. doi:10.1145/3768989
- [28] Saman Zadtootaghaj, Steven Schmidt, Nabajeet Barman, Sebastian Möller, and Maria G. Martini. 2018. A Classification of Video Games based on Game Characteristics linked to Video Coding Complexity. In *2018 16th Annual Workshop on Network and Systems Support for Games (NetGames)*. 1–6. doi:10.1109/NetGames.2018.8463434
- [29] Yasir Zaki, Thomas Pötsch, Jay Chen, Lakshminarayanan Subramanian, and Carmelita Görg. 2015. Adaptive Congestion Control for Unpredictable Cellular Networks. *SIGCOMM Comput. Commun. Rev.* 45, 4 (Aug. 2015), 509–522. doi:10.1145/2829988.2787498
- [30] Huanhuan Zhang, Anfu Zhou, Yuhuan Hu, Chaoyue Li, Guangping Wang, Xinyu Zhang, Huadong Ma, Leilei Wu, Aiyun Chen, and Changhui Wu. 2021. Loki: improving long tail performance of learning-based real-time video adaptation by fusing rule-based models. In *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking (New Orleans, Louisiana) (MobiCom '21)*. Association for Computing Machinery, New York, NY, USA, 775–788. doi:10.1145/3447993.3483259
- [31] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. 2018. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric. arXiv:1801.03924 [cs.CV] <https://arxiv.org/abs/1801.03924>
- [32] Xiaoqing Zhu and Rong Pan. 2013. NADA: A Unified Congestion Control Scheme for Low-Latency Interactive Video. In *2013 20th International Packet Video Workshop*. 1–8. doi:10.1109/PV.2013.6691448